
Sistemas Operacionais

Aula 02 - Gestão de Tarefas

Rogério Pereira Junior

Instituto Federal de Santa Catarina
campus São José
rogeriopereirajunior@gmail.com

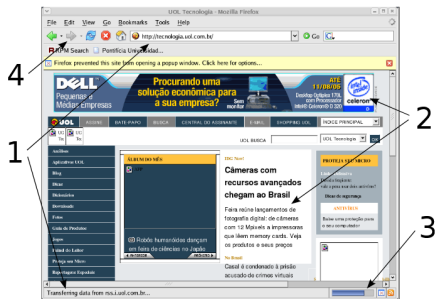
31 de março de 2026



- Definição de um sistema distribuídos
- Principais aspectos de sistemas distribuídos (exemplos)
- Motivação e principais desafios
- Tarefa



- Sistemas de computação executam, em geral, **mais tarefas do que processadores disponíveis**.
- frequente a necessidade de executar várias tarefas distintas simultaneamente
 - Editando uma imagem, impressão de telat´roio, escutando música, tudo ao mesmo tempo
 - Servidor de emails com milhares de usuários conectados remotamente enviam e recebem
 - Navegador Web executa várias tarefas em sua execução



Como atender a demanda dessas tarefas?



- Um processador convencional executa **apenas um fluxo de instruções por vez**.
- O sistema operacional deve:
 - Multiplexar o uso do processador;
 - Atender tarefas com diferentes necessidades;
 - Garantir eficiência, justiça e responsividade.
- A gestão de tarefas é um dos **papéis centrais do sistema operacional**.

nem sempre a capacidade de processamento existente é suficiente para atender a todos!



Tarefa



- Uma **tarefa** (task) é a execução de um fluxo sequencial de instruções.
- Conjunto de instruções carregadas na memória
- A tarefa é criada para atingir um objetivo específico:
 - realizar cálculos;
 - editar dados;
 - processar eventos;
 - interagir com usuários ou dispositivos.
- Trata-se de um conceito:
 - **Dinâmico**;
 - Com estado interno definido;
 - Que evolui ao longo do tempo.



■ Programa:

- Conjunto estático de instruções;
- Armazenado em disco;
- Não possui estado de execução;
- Exemplo: `/usr/bin/vi`, `notepad.exe`.

■ Tarefa:

- Execução do programa pelo processador;
- Possui estado interno (variáveis, posição do código);
- Interage com usuário, dispositivos e outras tarefas.

- Um mesmo programa pode originar **várias tarefas**.



Analogia: Receita e Execução

- Programa → receita em um livro.
- Disco → estante da cozinha.
- Computador → cozinha.



- Processador → cozinheiro.
- Tarefa → execução da receita.

Quais tarefas temos na execução da receita?



Como gerenciar as tarefas do sistema?

- O sistema operacional é responsável por:
 - organizar as tarefas;
 - decidir a ordem de execução;
 - controlar o uso do processador.
- As tarefas diferem em:
 - duração;
 - comportamento;
 - prioridade;
 - necessidade de E/S.
- A evolução da gerência de tarefas acompanha a evolução do hardware.

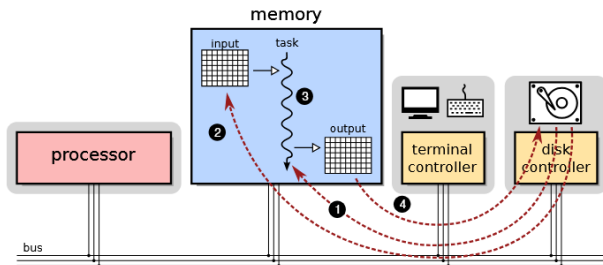
		4%	45%	5%	0%	
		CPU	Memória	Disco	Rede	
Nome	Status					
Aplicativos (6)						
>	Ferramenta de Captura		0%	53,6 MB	0 MB/s	0 Mbps
>	Firefox (24)		0,1%	2.953,7 MB	0,1 MB/s	0,1 Mbps
>	Gerenciador de Tarefas		1,1%	70,8 MB	0,6 MB/s	0 Mbps
>	Google Chrome (25)		0,9%	629,3 MB	0 MB/s	0 Mbps
>	Microsoft Teams (10)		0%	193,8 MB	0 MB/s	0 Mbps
>	Visual Studio Code (12)		0,6%	1.829,1 MB	0,1 MB/s	0 Mbps
Processos em segundo plano ...						
>	Ações do Aplicativo		0%	2,3 MB	0 MB/s	0 Mbps
>	Acrobat Collaboration Synchr...		0%	9,8 MB	0 MB/s	0 Mbps



- Executam **uma única tarefa por vez.**

- Fluxo típico:

- 1 carregar código;
- 2 carregar dados;
- 3 executar;
- 4 gravar resultados.

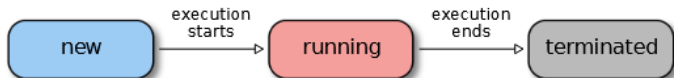


- Todas as etapas eram coordenadas manualmente.
- Usados principalmente para cálculos científicos e militares.
- Baixa eficiência no uso do processador.



■ Estados possíveis:

- **Nova:** A tarefa deixa de ser apenas um arquivo no disco (estático) e o SO começa a prepará-lo
- **Executando:** Aqui, a tarefa tem a posse total do processador.
- **Terminada:** O programa encerra sua execução.



- Não há concorrência entre tarefas.
- O processador fica ocioso durante operações de E/S.
 - O tempo de espera pode ser longo, já que dispositivos de E/S são muito mais lentos que a CPU.

Qual a consequência?

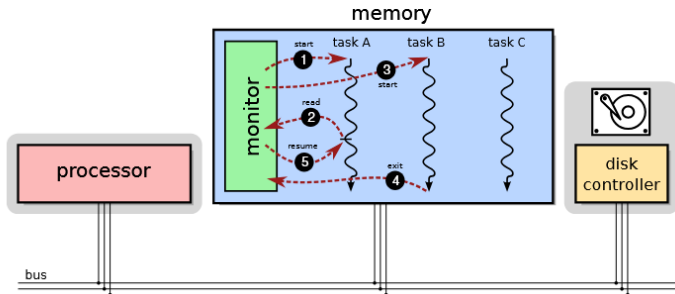


Sistemas Multitarefa

- Permitem suspender uma tarefa e executar outra -> [Troca de Contexto](#).

- Objetivo principal:

- evitar ociosidade do processador;
- aumentar produtividade do sistema.



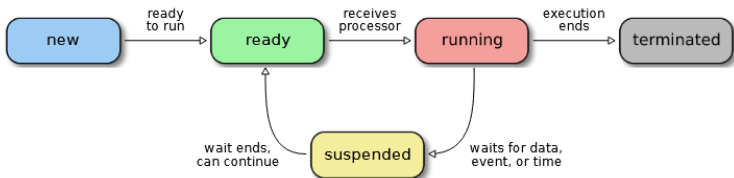
- Tarefas podem estar:

- executando;
- prontas;
- suspensas (aguardando eventos).

- Introdução da concorrência controlada.



- Carregar várias tarefas na memória
- Usar o processador ocioso para tratar outras tarefas
- Um software monitor coordena a troca de tarefas

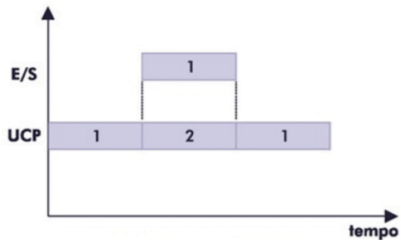


Suspensa/Bloqueada (Waiting)

A tarefa não pode rodar agora, mesmo que o processador esteja livre. Ela está esperando algo externo (um clique do mouse, um dado do HD, um pacote da internet).



Monotarefa vs Multitarefa



- Várias tarefas compartilham recursos (CPU, memória, dispositivos de E/S) e o sistema operacional precisa coordenar sua execução.



- Discutiremos melhor mais tarde



- Aplicações em laço infinito podem bloquear o sistema
- Aplicações interativas não funcionam bem

```
void main ()
{
    int i = 0, soma = 0 ;

    while (i < 1000)
        soma += i ;

    printf ("A soma vale %d\n", soma);
}
```

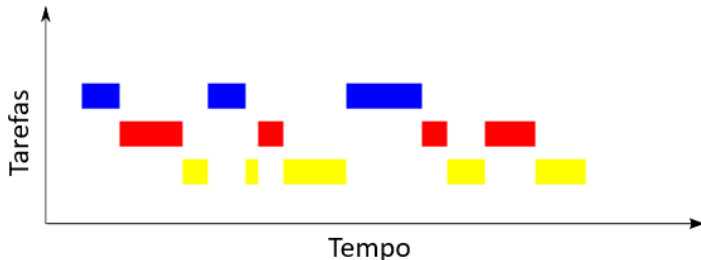


- **Percebe-se que temos uma fila que precisa ser gerencia no caso de sistemas multirefafa**
 - Em sistemas monotarefa uma fila com Capacidade igual 1 e consequentemente concorrência igual a zero.
- Cada tarefa recebe uma **fatia de tempo** (quantum).
 - Quantum típico vai de 10 ms a 200 ms
- Ao esgotar o quantum:
 - a tarefa perde o processador;
 - outra tarefa é escalonada.
- O sistema torna-se:
 - preemptivo;
 - interativo;
 - mais justo.



Time-Sharing (Tempo Compartilhado)

- Várias tarefas "vivas" ao mesmo tempo



- O tempo compartilhado é a estratégia de divisão que o Sistema Operacional usa para que todas elas pareçam estar rodando simultaneamente.



■ Concorrência

- É o termo geral
- Múltiplas tarefas disputando os mesmos recursos (CPU, RAM).
- controlar o uso do processador.

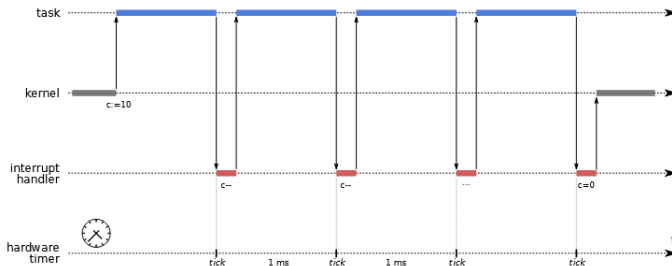
■ Tempo Compartilhado

- É o método específico para gerenciar essa concorrência em sistemas interativos.
- O foco aqui é o Tempo de Resposta.



Preempção por Tempo

- Implementada com temporizador de hardware.
- O temporizador gera interrupções periódicas (ticks).



- A cada tick:
 - o contador de quantum é decrementado;
 - ao chegar a zero ocorre a preempção.
- Evita monopolização do processador.



Ciclo de Vida das Tarefas

■ Estados principais:

■ Nova

- A tarefa está sendo criada
- Seu código está sendo carregado em memória, junto com as bibliotecas necessárias

■ Pronta

- A tarefa está em memória, pronta para iniciar ou retomar sua execução
- Todas as tarefas prontas são organizadas em uma fila
- Ordem determina por algoritmos de escalonamento

■ Executando;

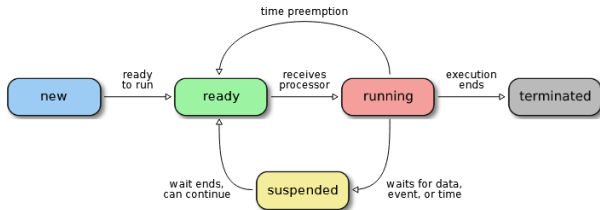
- O processador está dedicado à tarefa, executando suas instruções e fazendo avançar seu estado.

■ Suspensa;

- A tarefa não pode executar porque depende de dados externos ainda não disponíveis

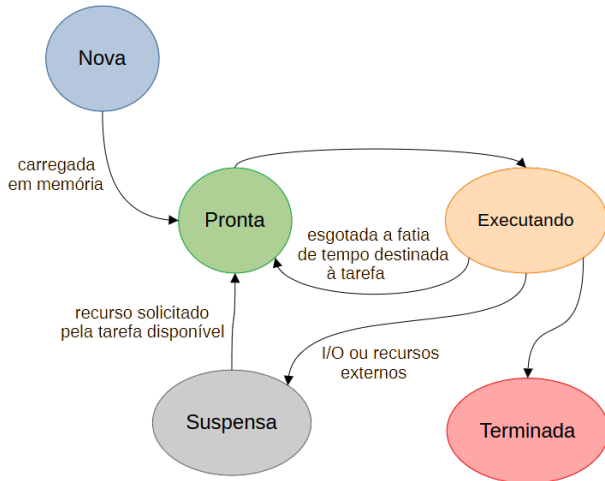
■ Terminada.

- O processamento da tarefa foi encerrado e ela pode ser removida da memória do sistema.



Ciclo de Vida das Tarefas - Transições

Admissão de uma nova tarefa e sua preparação



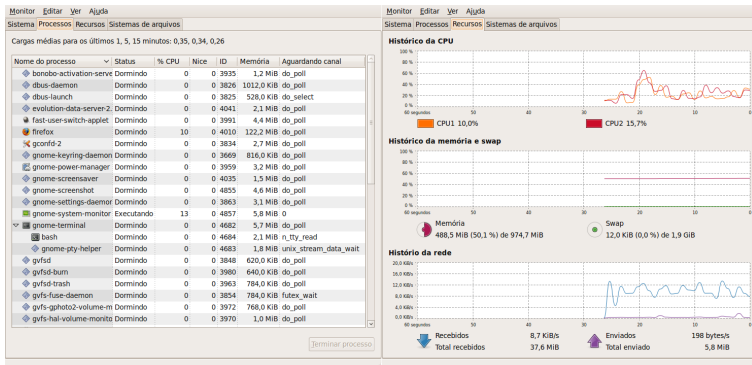
- $\dots \rightarrow N$ a tarefa ingressa no sistema
- $N \rightarrow P$ a tarefa está pronta para executar
- $P \rightarrow E$ a tarefa é escolhida para executar
- $E \rightarrow P$ esgota a fatia de tempo da tarefa
- $E \rightarrow T$ a tarefa encerra sua execução
- $T \rightarrow \dots$ a tarefa terminada é removida da memória
- $E \rightarrow S$ a tarefa em execução decide aguardar um recurso ou evento externo
- $S \rightarrow P$ o recurso ou evento aguardado pela tarefa está disponível



Prática



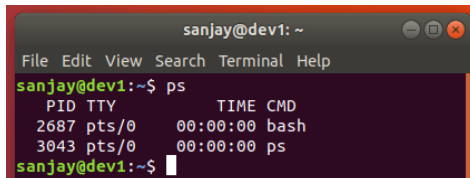
- Podemos analisar graficamente
- No Ubuntu, procure pelo programa **Monitor do sistema**



- Cada tarefa sendo executado no Linux está associado a um número inteiro positivo que é conhecido como PID (Process IDentification).



- O comando **ps aux** gera uma “fotografia” das tarefas rodando na máquina
- O comando **ps** exibe apenas os tarefas sendo executados no terminal.



A terminal window titled 'sanjay@dev1: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The prompt is 'sanjay@dev1:~\$'. The command 'ps' has been entered, and the output is displayed as a table with columns PID, TTY, TIME, and CMD.

PID	TTY	TIME	CMD
2687	pts/0	00:00:00	bash
3043	pts/0	00:00:00	ps

The prompt is now 'sanjay@dev1:~\$' with a cursor.

- **ps -gl** lista de todos os terminais caso tenha mais de um em aberto
- **ps -A** mostra todos os processo.
- **ps -u** fornece o nome do usuário e a hora de início do processo.
- Mais informações do comando ps - [Acesse aqui](#)
- Podemos usar também o **top**. Neste caso haverá atualização periódica da tela, fazendo uma amostra on-line tarefas ativos.



- PID – ID do processo.
- USER - Usuário proprietário do processo.
- PR - A prioridade de agendamento do processo.
 - Valor: 0-20
 - Quanto menor o valor, maior a prioridade
- NI - O NICE é um atributo que permite influenciar a prioridade do processo.
 - $PR = 20 + NI$
 - Podemos alterar o NI para a faixa de -20 até 19
- VIRT - A quantidade de memória virtual usada pelo processo.

```
Cpu(s): 21.5%us, 5.6%sy, 0.0%ni, 63.7%id, 9.2%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 3058052k total, 2083212k used, 974840k free, 167964k buffers
Swap: 3481788k total, 0k used, 3481788k free, 1090496k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2398	ricardo	20	0	428m	76m	29m	S	15	2.6	2:48.27	compiz
5489	ricardo	20	0	169m	14m	10m	S	9	0.5	0:00.28	gnome-screensho
3187	ricardo	20	0	1042m	393m	42m	S	8	13.2	27:00.38	firefox
1238	root	20	0	92920	26m	8724	S	7	0.9	4:22.79	Xorg



Tarefas - Componente de um tarefa

- RES - O tamanho da memória usada. Residente na memória física
- SHR - é a memória compartilhada usada pelo tarefa.
- S – estado: D ou I (ininterrupto), R (executando), S (dormindo), T (rastreado ou parado), Z (zumbi)
- CPU - É a porcentagem de tempo de CPU que a tarefa tem usado desde a última atualização.
- MEM - Percentagem de memória física disponível usada pelo tarefa.
- TEMPO - O tempo total de CPU que a tarefa tem usado desde o início (precisão de centésimo de segundo)
- COMANDO - Descrição do comando que foi utilizado para iniciar o tarefa.

```
Cpu(s): 21.5%us, 5.6%sy, 0.0%ni, 63.7%id, 9.2%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 3058052k total, 2083212k used, 974840k free, 167964k buffers
Swap: 3481788k total, 0k used, 3481788k free, 1090496k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2398	ricardo	20	0	428m	76m	29m	S	15	2.6	2:48.27	compiz
5489	ricardo	20	0	169m	14m	10m	S	9	0.5	0:00.28	gnome-screensho
3187	ricardo	20	0	1042m	393m	42m	S	8	13.2	27:00.38	firefox
1238	root	20	0	92920	26m	8724	S	7	0.9	4:22.79	Xorg



Opções do comando

- Tempo de atualização (a cada n segundos): **top -d n**
- Especificando o usuário: **top -u "usuario"**
- Especificando um tarefa específico: **top -p pid**

```
Cpu(s): 21.5%us, 0.0%sy, 0.0%ni, 63.7%id, 9.2%wa, 0.0%hi, 0.0%si, 0.0%st  
Mem: 3058052k total, 2083212k used, 974840k free, 167964k buffers  
Swap: 3481788k total, 0k used, 3481788k free, 1090496k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2398	ricardo	20	0	428m	76m	29m	S	15	2.6	2:48.27	compiz
5489	ricardo	20	0	169m	14m	10m	S	9	0.5	0:00.28	gnome-screensho
3187	ricardo	20	0	1042m	393m	42m	S	8	13.2	27:00.38	firefox
1238	root	20	0	92920	26m	8724	S	7	0.9	4:22.79	Xorg

- Mais informações do comando top - [Acesse aqui](#)



- **ps aux** : vemos uma “fotografia” dos tarefas rodando na máquina
- **ps** (sem opções): exibe apenas os tarefas sendo executados no terminal.
- **ps -A**: mostra todos os tarefas
- **ps -Al**: mostra todos os tarefas de forma detalhada
- **ps -gl**: lista de todos os terminais caso tenha mais de um em aberto
- Mais sobre o comando **ps** - [Acesse aqui](#)
- **top**: visualiza-se os tarefas com o maior uso do processador
- **top -d n**: Tempo de atualização (a cada n segundos)
- **top -u “usuario”**: Especificando o usuário
- **top -p pid**: Visualizando um tarefa específico



- Listar todos os arquivos abertos
 - **sudo lsof**
- Listar arquivos abertos de um usuário específico
 - **sudo lsof -u usuário**
- Listar tarefas em uma porta específica
 - **sudo lsof -i TCP:1-1024**
- Listar todas as conexões de rede
 - **sudo lsof -i**



Interrompendo a execução de um tarefa

- Nos podemos matar um tarefa que esteja rodando utilizando o atalho **CTRL C**
- Para exemplificar digitem no shell **pluma teste.txt**. Neste momento o editor de texto pluma ira abrir.
- Volte para o terminal, perceba que ele não pode ser utilizado visto que esta rodando o **tarefa** do editor pluma em **primeiro plano**
- Tecle **CTRL C**, imediatamente o editor é fechado, isto é, o tarefa relativo ao programa pluma foi "morto".



Parando momentaneamente a execução de um tarefa

- Do mesmo modo que podemos "matar" um tarefa, pode-se suspender um tarefa por um tempo
- Para isto basta teclar **CTRL Z** em um tarefa rodando em seu terminal
- Para exemplificar, abram novamente o editor de texto: **pluma teste.txt**
- Volte ao terminal e tecele **CTRL Z**
- O tarefa é suspenso, você não consegue digitar nada dentro do documento



- **Primeiro Plano**- Quando você deve esperar o término da execução de um programa para executar um novo comando.
- **Segundo Plano** - Quando você não precisa esperar o término da execução de um programa para executar um novo comando.
- Após iniciar um programa em background, é mostrado um número PID (identificação do tarefa) e o aviso de comando é novamente mostrado, permitindo o uso normal do sistema.
- Para iniciar um programa em primeiro plano, basta digitar seu nome normalmente.
- Para iniciar um programa em segundo plano, acrescente o carácter **&** após o final do comando.
 - **pluma &**
 - **firefox &**
 - **gparted &**



- **jobs**: mostra os tarefas que estão parados ou rodando em **segundo plano**.



- O comando **kill** envia sinais de controle para tarefas
- Cada flag representa um sinal
- Para matarmos um tarefa em execução usamos o comando **kill -9** seguido do número do tarefa (PID).
- **kill -9 PID**: matarmos o tarefa em execução com o PID específico
 - **kill -9 1234**: matarmos o tarefa em execução com o PID 1234 .



- Pausar/Suspender um tarefa
 - **kill -STOP PID**
- Retomar um tarefa suspenso
 - **kill -CONT PID**

- Temos também o comando **killall** no qual usa o nome do tarefa no lugar do **PID**
- Ou seja, mata-se todos os tarefas abertos pelo comando específico
- Ex: **killall pluma**



- O comando **Nice** permite que você priorize tarefas no caso de você estar executando muitos deles em seu sistema.
- A prioridade de execução de uma tarefa normal pode variar de 0 (maior prioridade) a 39 (menor prioridade).
- $PR = 20 + NI$
- O NI pode variar de -20 a 19.
- Para diminuir a prioridade
 - **nice -n 6 pluma**: o programa é executado com $NI = 6$ e $PR = 26$.
 - **nice -n 15 zip -r docBackup.zip /home/aluno/Documentos** o programa é executado com $NI = 15$ e $PR = 35$.
- Para aumentar a prioridade
 - **sudo nice -n -6 pluma**: o programa é executado com $NI = -6$ e $PR = 14$.

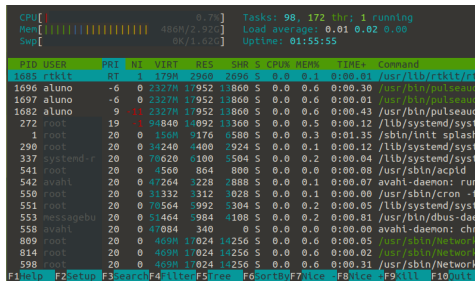


- O comando **renice** modifica a prioridade de um tarefa já em execução
- **renice -p pid** : altera a prioridade do tarefa que possui o PID especificado. É o padrão
 - **sudo renice '+10' -p PID**: Diminuindo a prioridade
 - **sudo renice '-10' -p PID**: Aumentando a prioridade



Comando htop

- **htop**: auxilia o usuário a monitorar de forma interativa e em tempo real os recursos de seu sistema operacional Linux.
- **sudo apt install htop**



The screenshot shows the htop interface with the following statistics at the top:

- CPU: 0.7%
- Mem: 486M/2.92G
- Swap: 0K/1.02G
- Tasks: 98, 172 thr; 1 running
- Load average: 0.01 0.02 0.00
- Uptime: 01:55:55

Below the statistics is a table of running processes:

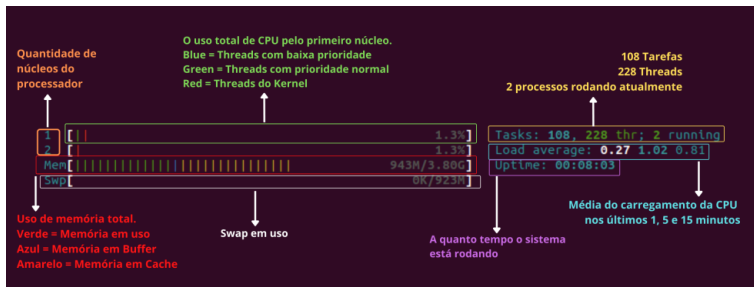
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
1685	rtkit	RT	1	179M	2968	2696	S	0.0	0.1	0:00.01	/usr/lib/rtkit/rt
1696	aluno	-6	0	2327M	17952	13860	S	0.0	0.6	0:00.30	/usr/bin/pulseaud
1697	aluno	-6	0	2327M	17952	13860	S	0.0	0.6	0:00.01	/usr/bin/pulseaud
1682	aluno	9	-11	2327M	17952	13860	S	0.0	0.6	0:00.43	/usr/bin/pulseaud
272	root	19	-1	94840	14092	13360	S	0.0	0.5	0:00.12	/lib/systemd/syst
1	root	20	0	156M	9176	6580	S	0.0	0.3	0:01.35	/sbin/init splash
290	root	20	0	34240	4400	2924	S	0.0	0.1	0:00.12	/lib/systemd/syst
337	systemd-r	20	0	70620	6100	5504	S	0.0	0.2	0:00.04	/lib/systemd/syst
541	root	20	0	4560	864	800	S	0.0	0.0	0:00.08	/usr/sbin/acpid
542	avahi	20	0	47264	3228	2888	S	0.0	0.1	0:00.07	avahi-daemon: run
550	root	20	0	31332	3312	3028	S	0.0	0.1	0:00.00	/usr/sbin/cron -f
551	root	20	0	70564	5992	5304	S	0.0	0.2	0:00.05	/lib/systemd/syst
553	messagebu	20	0	51464	5984	4108	S	0.0	0.2	0:00.81	/usr/bin/dbus-dae
558	avahi	20	0	47084	340	0	S	0.0	0.0	0:00.00	avahi-daemon: chr
809	root	20	0	469M	17024	14256	S	0.0	0.6	0:00.05	/usr/sbin/Network
814	root	20	0	469M	17024	14256	S	0.0	0.6	0:00.02	/usr/sbin/Network
598	root	20	0	469M	17024	14256	S	0.0	0.6	0:00.31	/usr/sbin/Network

At the bottom, there is a legend for keyboard shortcuts: F1Help, F2Setup, F3Search, F4Filter, F5Tree, F6SortBy, F7Nice, F8Icc, F9All, F10Quit.

- **F3**: Procurar um PID ou nome específico
- **F6**: Ordenação dos registros capturados
- **F7**: Aumentar a prioridade
- **F8**: Diminuir a prioridade
- **F9**: Matar o tarefa
- **F10**: Sair



Comando htop

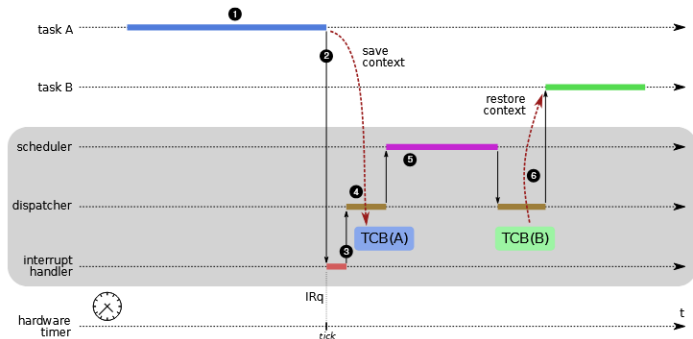


Implementação de tarefas



Troca de Contexto

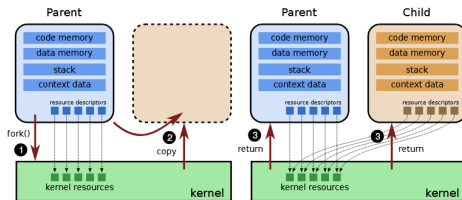
- Ocorre quando o SO suspende uma tarefa e ativa outra.
- Envolve:
 - salvar o contexto da tarefa atual;
 - restaurar o contexto da próxima tarefa.



- Implementada em baixo nível (linguagem de máquina).
- Possui custo computacional.



No caso dos sistemas UNIX, a criação de novos processos é feita em duas etapas: na primeira etapa, um processo cria uma réplica de si mesmo, usando a chamada de sistema `fork()`. Todo o espaço de memória do processo inicial (pai) é copiado para o novo processo (filho), incluindo o código da(s) tarefa(s) associada(s) e os descritores dos arquivos e demais recursos associados ao mesmo. A Figura 5.3 ilustra o funcionamento dessa chamada.



A chamada de sistema `fork()` é invocada por um processo, mas dois processos recebem seu retorno: o processo pai, que a invocou, e o processo filho, que acabou de ser criado e que possui o mesmo estado interno que o pai (ele também está aguardando o retorno da chamada de sistema). Ambos os processos aces

copia: variáveis copiadas, ou seja, são diferentes e evoluem de maneira diferente



- Após o `fork()`, ambos os processos continuam a execução a partir da mesma instrução.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();
    printf("Oi!\n");
    return 0;
}
```

- O que acontece?
 - 1 processo vira 2
 - Ambos executam o `printf`

Analogia

- Imagine um documento sendo duplicado:
 - Você (pai) continua escrevendo
 - Sua cópia (filho) também continua
 - Ambos começam do mesmo ponto

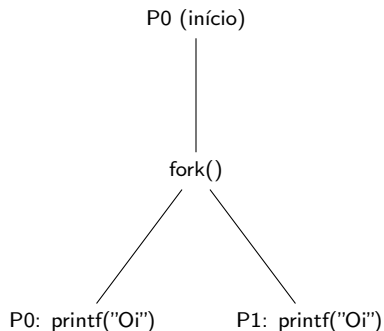


Fluxo do fork()

- 1 P0 inicia
- 2 chama fork()
- 3 cria P1
- 4 continua
- 5 continua
- 6 Ambos executam printf

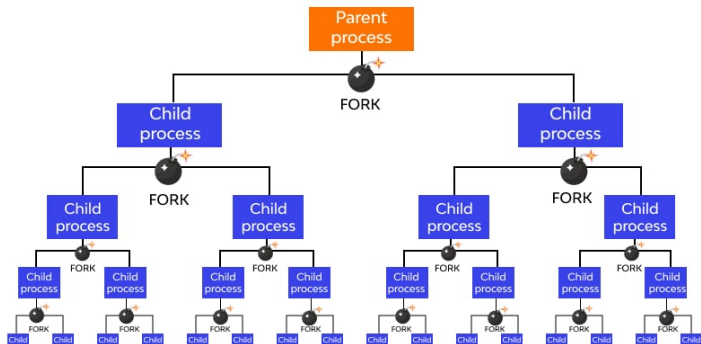
```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();
    printf("Oi!\n");
    return 0;
}
```



Explosão de Processos

- Ocorre quando um programa cria processos em grande quantidade
- Geralmente usando múltiplas chamadas de `fork()`
- O crescimento é **exponencial**



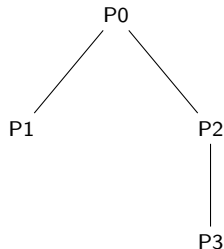
- Cada `fork()` duplica os processos
- Fórmula: 2^n



- 2^n
- $2^2 = 4$ processos

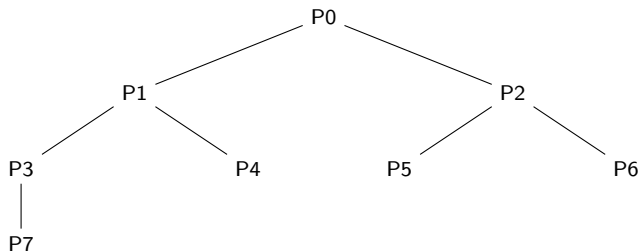
```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();
    fork();
    printf("Oi\n");
    return 0;
}
```



Explosão de processos (3 forks)

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    fork();
    fork();
    printf("0i\n");
    return 0;
}
```



■ Total: 8 processos (2^3)



- Exibe os processos em execução no formato de uma árvore hierárquica, facilitando a visualização das relações pai-filho entre eles.

```
[root@host ~]# pstree -p
systemd(1)─NetworkManager(1308)─{NetworkManager}(1332)
                                └─{NetworkManager}(1336)
    ─agetty(1327)
    ─atop(9792)
    ─auditd(1242)─{auditd}(1243)
    ─crond(32113)
    ─dbus-daemon(1287)
    ─irqbalance(32354)
    ─lvmetad(845)
    ─master(363)─bounce(22284)
                ├──cleanup(17834)
                ├──local(17307)
                ├──pickup(17303)
                ├──qmgr(365)
                ├──showq(23680)
                └─trivial-rewrite(21826)
    ─mysqld(32450)─{mysqld}(32462)
                 ├──{mysqld}(32463)
                 ├──{mysqld}(32464)
                 ├──{mysqld}(32465)
                 ├──{mysqld}(32466)
                 ├──{mysqld}(32467)
                 ├──{mysqld}(32468)
                 └─{mysqld}(32469)
```



Threads



Threads

São as menores unidades de processamento que um sistema operacional pode gerenciar, representando fluxos sequenciais de instruções dentro de um processo.

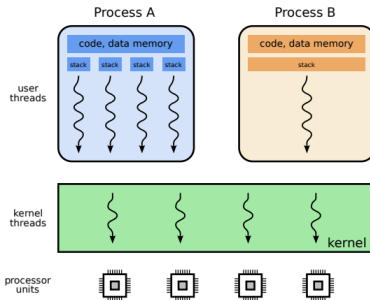


Figura 5.4: *Threads* de usuário e de núcleo.

- Um processo pode ter partes diferentes do seu código sendo executadas concorrentemente, com um menor overhead do que utilizando múltiplos processos
- Diferente dos processos, threads de um mesmo processo compartilham código, dados e arquivos.



- Thread é um fluxo de execução independente.
- Um processo pode conter várias threads.
- Threads compartilham:
 - memória;
 - arquivos;
 - recursos do processo.
- Cada thread possui:
 - pilha própria;
 - registradores próprios.

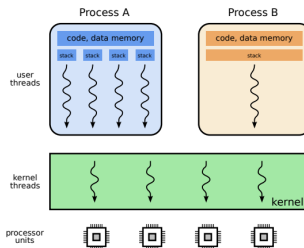


Figura 5.4: Threads de usuário e de núcleo.

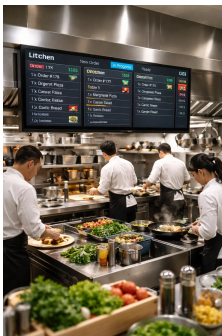


- Processo: É a instância de um programa em execução, com sua própria memória protegida.
- Thread: É o "trabalhador" dentro do processo que realiza as tarefas. Múltiplas threads podem trabalhar juntas em um processo
 - forma de um processo se autodividir em duas ou mais tarefas.

Pense numa thread como uma sequência de comandos sendo executados em um programa. Se você tiver duas threads, terá duas sequências de comandos executando ao mesmo tempo no mesmo programa ou processo.



- Imagine que o seu Processo é o Restaurante inteiro.
- Ela possui
 - espaço (endereço de memória)
 - equipamentos (recursos do sistema)
 - estoque (dados)
- Um restaurante isolado não compartilha o fogão ou a despensa com o restaurante vizinho da rua de trás.
- As threads são os cozinheiros dentro desse mesmo restaurante.
 - Compartilhamento de Recursos
 - Economia (Leveza)
 - Concorrência



- Temos um risco: **Condição de Corrida**
- Se dois cozinheiros tentarem usar a mesma boca do fogão ao mesmo tempo sem se comunicarem, haverá um problema.
- Se um cozinheiro tempera a sopa e, logo em seguida, outro tempera a mesma sopa sem saber, ela ficará salgada demais.
- Precisamos de **sincronização**!

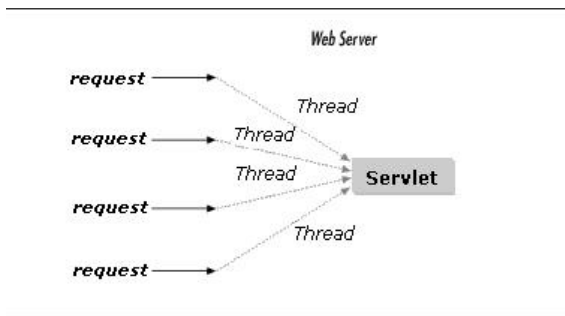


- Aplicações modernas realizam várias atividades simultâneas:
 - interfaces gráficas;
 - comunicação em rede;
 - processamento em segundo plano.
- Usar um processo por atividade é custoso.
- Threads permitem:
 - maior paralelismo lógico;
 - compartilhamento eficiente de recursos;
 - menor custo de criação e troca de contexto.



Threads no Back-end: O Modelo de Requisição

- **Cenário:** Servidores (Tomcat, Apache) recebem milhares de conexões simultâneas.
- **Modelo Thread-per-Request:**
 - O processo principal "escuta" as portas (80/443).
 - Cada nova requisição HTTP é atribuída a uma **thread exclusiva** (criada na hora ou vinda de um *Pool*).
- **Vantagens Práticas:**
 - **Paralelismo:** Atendimento de múltiplos usuários simultâneos.
 - **Isolamento:** Dados da requisição associados à pilha (*stack*) da thread.
 - **Não-bloqueio:** Se a Thread A aguarda o Banco de Dados, a Thread B continua processando.



- Foco: "dividir para conquistar"
- Exemplo: Temos 1 milhão de linhas para processar
- Não processamos uma por uma em sequência.
- Em vez disso
 - Divide esse milhão de linhas em 4 blocos de 250 mil.
 - Cria 4 threads, e cada uma processa um bloco em um núcleo diferente da CPU.
 - No final juntamos os resultados
- Muitas vezes o processamento de dados envolve: Ler do disco -> Processar -> Salvar no Banco.
 - Enquanto a Thread 1 está salvando o resultado anterior no banco (tarefa lenta de I/O)
 - Thread 2 já está processando a próxima leva de dados (tarefa de CPU).

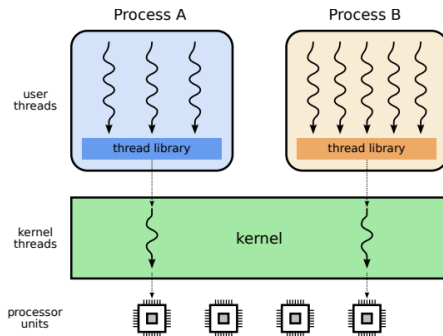


- Threads de usuário precisam ser gerenciadas pelo núcleo.
- Existem três modelos clássicos:
 - N:1 (many-to-one);
 - 1:1 (one-to-one);
 - N:M (many-to-many).
- Os modelos diferem em:
 - desempenho;
 - paralelismo;
 - complexidade;
 - escalabilidade.



Modelo de Threads N:1 (Many-to-One)

- Várias threads de usuário mapeadas em:
 - uma única thread de núcleo.
- Gerenciamento feito por:
 - bibliotecas em espaço de usuário.
- O núcleo enxerga apenas:
 - um fluxo de execução por processo.



Modelo de Threads N:1 (Many-to-One)

■ A Analogia da Linha Única:

- 10 ramais internos (Threads Usuário) para apenas 1 linha externa (Thread Núcleo).
- Os 10 funcionários podem conversar entre si livremente e passar recados (gerenciamento rápido no espaço do usuário)
- Porém, se alguém precisa falar com o mundo exterior (o Kernel/Processador), ele precisa ocupar a única linha disponível.



Limitação de Hardware

Mesmo em um computador com 16 núcleos de CPU, este modelo só consegue usar um núcleo por vez, pois só existe uma "autorização"(thread de núcleo) para rodar.



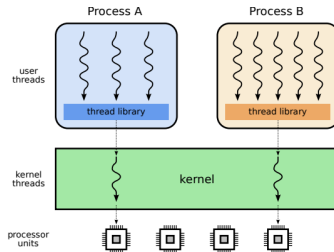
Vantagens e Desvantagens do Modelo N:1

■ Vantagens:

- baixo custo de criação;
- trocas de contexto rápidas;
- alta escalabilidade no espaço do usuário.

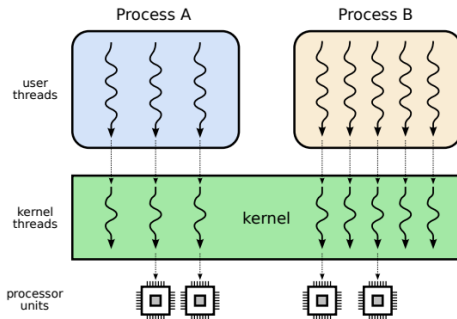
■ Desvantagens:

- chamadas bloqueantes bloqueiam todas as threads;
- não explora múltiplos processadores;
- divisão injusta de CPU entre threads.



Modelo de Threads 1:1 (One-to-One)

- Cada thread de usuário corresponde a:
 - uma thread de núcleo.
- Gerenciamento totalmente realizado pelo SO.
- Modelo adotado pela maioria dos sistemas atuais.
- Permite que várias threads rodem em CPUs diferentes simultaneamente de forma real.



Modelo de Threads 1:1 (One-to-One)

- Imagine um banco onde cada cliente que entra (1 thread de usuário) recebe um gerente exclusivo (1 thread de núcleo) para acompanhá-lo.
- Se o Cliente A precisar esperar o sistema do banco carregar (bloqueio), o Cliente B não é afetado, pois ele tem seu próprio gerente dedicado.



Desvantagem

Criar um "gerente" para cada cliente custa caro e consome memória. Se você criar 10.000 threads, o SO pode ficar sobrecarregado criando 10.000 entidades de núcleo.



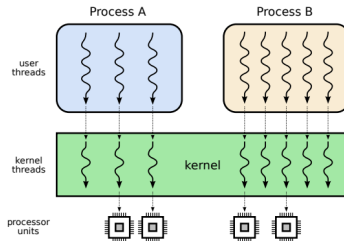
Vantagens e Desvantagens do Modelo 1:1

■ Vantagens:

- verdadeiro paralelismo;
- chamadas bloqueantes não afetam outras threads;
- escalonamento justo pelo núcleo.

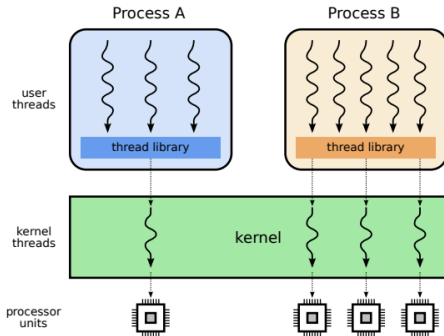
■ Desvantagens:

- maior custo de criação;
- sobrecarga no núcleo;
- baixa escalabilidade para milhões de threads.



Modelo de Threads N:M (Many-to-Many)

- Combina características dos modelos N:1 e 1:1.
- N threads de usuário mapeadas em:
 - M threads de núcleo ($M < N$).
- Utiliza:
 - biblioteca de threads;



Modelo de Threads N:M (Many-to-Many)

- Imagine um call center.
- Temos 10 atendentes virtuais no software (N threads de usuário),
- Porém apenas 4 linhas telefônicas físicas saindo do prédio (M threads de núcleo).
- O sistema alterna os atendentes nas linhas disponíveis. Se um atendente fica mudo, o supervisor (SO) simplesmente passa a linha física para outro atendente que tenha algo para falar.



Desvantagem

É o mais complexo de implementar, pois exige que o "supervisor" seja muito inteligente para orquestrar quem ganha a linha a cada milissegundo.



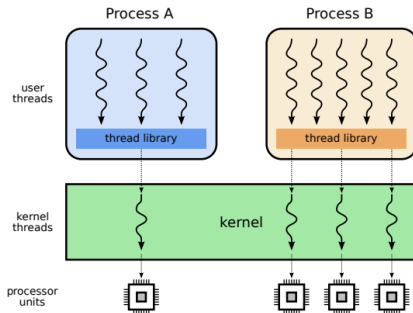
Vantagens e Desvantagens do Modelo N:M

■ Vantagens:

- bom paralelismo;
- melhor escalabilidade que 1:1;
- menor bloqueio global que N:1.

■ Desvantagens:

- implementação complexa;
- maior custo de gerência;



Comparação entre os Modelos de Threads

Modelo	N:1	1:1	N:M
Implementação	biblioteca	núcleo	ambos
Complexidade	baixa	média	alta
Custo de gerência	nulo	médio	alto
Escalabilidade	alta	baixa	alta
Vários processadores	não	sim	sim
Trocas de contexto	rápida	lenta	ambos
Divisão de recursos	injusta	justa	variável
Exemplos	GNU Threads Python	Windows Linux	Solaris FreeBSD



Threads são mais rápidas que processos?



Por que threads são perigosas?

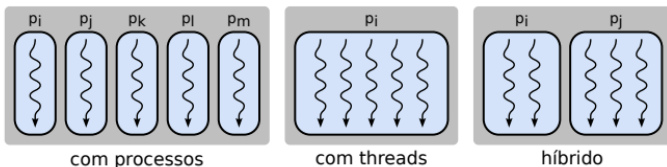


Threads no Linux



■ Processos

- Isolamento de memória → maior robustez
- Permite diferentes usuários/permissões
- Compartilhamento de dados exige mecanismos especiais (memória compartilhada, IPC)



■ Threads

- Compartilham espaço de endereçamento e recursos
- Mais simples para interação entre tarefas
- Criação mais rápida que processos
- Menor robustez: falha em uma thread afeta todo o processo

■ Modelo Híbrido

- Combina robustez dos processos com desempenho das threads
- Cada processo contém múltiplas threads



Uso de processos versus threads

Característica	Com processos	Com <i>threads</i> (1:1)	Híbrido
Custo de criação de tarefas	alto	baixo	médio
Troca de contexto	lenta	rápida	variável
Uso de memória	alto	baixo	médio
Compartilhamento de dados entre tarefas	canais de comunicação e áreas de memória compartilhada.	variáveis globais e dinâmicas.	ambos.
Robustez	alta, um erro fica contido no processo.	baixa, um erro pode afetar todas as <i>threads</i> .	média, um erro pode afetar as <i>threads</i> no mesmo processo.
Segurança	alta, cada processo pode executar com usuários e permissões distintas.	baixa, todas as <i>threads</i> herdam as permissões do processo onde executam.	alta, <i>threads</i> que necessitam as mesmas permissões podem ser agrupadas em um mesmo processo.
Exemplos	Servidor Apache (versões 1.*), SGBD PostgreSQL	Servidor Apache (versões 2.*), SGBD MySQL	Navegadores Chrome e Firefox, SGBD Oracle

